

FEM-PR: istruzioni per lo script del modello

1 Idea generale

Il progetto e' organizzato intorno a due classi principali:

- **Domain**: contiene gli oggetti del modello, divisi per categoria (`nodes`, `materials`, `sections`, `beamIntegrations`, `geometricTransformations`, `elements`).
- **Model**: fornisce una sintassi compatta per creare gli oggetti e registrarli nel **Domain**.

L'utente normalmente lavora con **Model**. Per esempio:

```
from model import Model

m = Model()

m.node(1, 0.0, 0.0)
m.node(2, 0.0, 3.0)
```

Ogni metodo di costruzione crea un oggetto, lo aggiunge al **Domain** e lo restituisce. Quindi sono equivalenti, come stile d'uso:

```
m.node(1, 0.0, 0.0)
```

oppure:

```
n1 = m.node(1, 0.0, 0.0)
```

Nel primo caso il nodo viene comunque salvato in `m.nodes`; nel secondo caso si conserva anche un riferimento locale all'oggetto.

2 Pattern prototipo e copia

Alcuni oggetti vengono definiti come prototipi nel **Domain**. Quando un altro oggetto ha bisogno di usarli con stato indipendente, viene creata una copia.

2.1 Materiali

I materiali definiti dall'utente sono salvati in `m.materials`. Le sezioni che hanno bisogno di materiali indipendenti usano copie dei materiali.

2.2 Sezioni

Le sezioni sono salvate in `m.sections`. Gli elementi possono creare copie delle sezioni, ad esempio una copia per ogni punto di integrazione.

2.3 Trasformazioni geometriche

La trasformazione geometrica e' un caso particolare. Nel modello si definisce solo il tipo e l'identificativo:

```
m.geomTransf("linear", "linearTransf")
```

La trasformazione diventa concreta solo quando viene assegnata a un elemento, perche' in quel momento sono noti `nodeI` e `nodeJ`. Il metodo `getGeomTransfCopy(...)` crea quindi una copia inizializzata sui nodi dell'elemento.

3 Sintassi dello script del modello

3.1 Creazione del modello

```
from model import Model  
  
m = Model()
```

3.2 Nodi

```
m.node(id, x, y)
```

- `id`: identificativo del nodo, intero o stringa.
- `x, y`: coordinate del nodo nel piano.

Esempio:

```
m.node(1, 0.0, 0.0)  
m.node(2, 0.0, 3.0)
```

3.3 Materiali

3.3.1 elastic

```
m.material("elastic", id, E, rho=0.0)
```

- `id`: identificativo del materiale.
- `E`: modulo elastico.
- `rho`: densita', opzionale.

Esempio:

```
m.material("elastic", "matElastic", 210000.0)
```

3.3.2 steel01

```
m.material("steel01", id, fy, E0, b, a1=0.07, a2=2, a3=0.07, a4=2, rho=0.0)
```

- id: identificativo del materiale.
- fy: tensione di snervamento.
- E0: modulo elastico iniziale.
- b: rapporto di incrudimento.
- a1, a2, a3, a4: parametri opzionali del modello.
- rho: densita', opzionale.

Esempio:

```
m.material("steel01", "mat1", 450.0, 210000.0, 0.005)
```

3.4 Sezioni

3.4.1 elasticSection

```
m.section("elasticSection", id, E, A, I)
```

- id: identificativo della sezione.
- E: modulo elastico.
- A: area.
- I: momento di inerzia.

Esempio:

```
m.section("elasticSection", "secElastic", 210000.0, 0.02, 8.0e-5)
```

3.4.2 aggregator

```
m.section("aggregator", id, axialID, bendingID)
```

- id: identificativo della sezione.
- axialID: identificativo del materiale per la risposta assiale.
- bendingID: identificativo del materiale per la risposta flessionale.

Esempio:

```
m.material("steel01", "mat1", 450.0, 210000.0, 0.005)  
m.section("aggregator", "sec1", "mat1", "mat1")
```

3.5 Integrazione lungo l'elemento

3.5.1 lobatto

```
m.beamIntegration("lobatto", id, nIP, sectionID)
```

- id: identificativo della regola di integrazione.
- nIP: numero di punti di integrazione.
- sectionID: identificativo della sezione associata.

Esempio:

```
m.beamIntegration("lobatto", "beamInt1", 5, "sec1")
```

3.5.2 legendre

```
m.beamIntegration("legendre", id, nIP, sectionID)
```

Esempio:

```
m.beamIntegration("legendre", "beamInt2", 3, "sec1")
```

3.6 Trasformazioni geometriche

3.6.1 linear

```
m.geomTransf("linear", id)
```

- id: identificativo della trasformazione geometrica.

Esempio:

```
m.geomTransf("linear", "linearTransf")
```

3.7 Elementi

3.7.1 dispBeamColumn

```
m.element("dispBeamColumn", id, nodeI_ID, nodeJ_ID, geomTransfID,  
          beamIntID)
```

- id: identificativo dell'elemento.
- nodeI_ID: identificativo del nodo iniziale.
- nodeJ_ID: identificativo del nodo finale.
- geomTransfID: identificativo della trasformazione geometrica.
- beamIntID: identificativo della regola di integrazione.

Esempio:

```
m.element("dispBeamColumn", "dispBC", 1, 2, "linearTransf", "beamInt1")
```

Quando l'elemento viene creato, il modello:

1. recupera i due nodi dal Domain;
2. crea una copia della trasformazione geometrica inizializzata sui due nodi;
3. crea una copia della `beamIntegration`;
4. costruisce l'elemento `DispBeamColumn`;
5. aggiunge l'elemento a `m.elements`.

All'interno di `DispBeamColumn`, il metodo `initialize()` crea una copia della sezione per ogni punto di integrazione e inizializza vettori e matrici dell'elemento.

4 Esempio completo

```
from model import Model

m = Model()

m.node(1, 0.0, 0.0)
m.node(2, 0.0, 3.0)

m.material("steel01", "mat1", 450.0, 210000.0, 0.005)
m.section("aggregator", "sec1", "mat1", "mat1")

m.beamIntegration("lobatto", "beamInt1", 5, "sec1")
m.geomTransf("linear", "linearTransf")

ele = m.element("dispBeamColumn", "dispBC", 1, 2, "linearTransf", "
    beamInt1")

print(m.nodes)
print(m.materials)
print(m.sections)
print(m.beamIntegrations)
print(m.geometricTransformations)
print(m.elements)
```

5 Metodi espliciti disponibili

Oltre alla sintassi compatta, `Model` espone anche metodi espliciti:

```
m.addElasticMaterial(id, E, rho=0.0)
m.addSteel01Material(id, fy, E0, b, a1=0.07, a2=2, a3=0.07, a4=2, rho
    =0.0)

m.addElasticSection(id, E, A, I)
m.addAggregator(id, axialID, bendingID)

m.addGaussLobatto(id, nIP, sectionID)
m.addGaussLegendre(id, nIP, sectionID)
```

```
m.addLinearGeomTransf(id)
m.addDispBeamColumn(id, nodeI_ID, nodeJ_ID, geomTransfID, beamIntID)
```

Questi metodi sono piu' espliciti e possono essere comodi durante debug, documentazione o uso con suggerimenti dell'IDE.